



“ Formando **nuevas generaciones** con **sello de excelencia** comprometidos
con la **transformación social** de las regiones y un país en paz ”

PROYECTO DE INGENIERÍA DE SOFTWARE

Plato Solidario

Plataforma Tecnológica de Logística Social y Redistribución de Almuerzos Universitarios

Estefany Dayana Vela Rangel

C.C. 1193213518

Facultad de Ingenierías y
Arquitectura Programa de
Ingeniería de Software

**Universidad de
Pamplona**

2026



1. Introducción y Marco de Trabajo

1.1. Objetivos del Proyecto

Objetivo General

Diseñar y especificar una plataforma tecnológica integral (aplicación móvil y panel web) de logística social y distribución en tiempo real que intercepte y redistribuya raciones de almuerzo previamente adquiridas por estudiantes que no pueden reclamarlas un día determinado, entregándolas a estudiantes que ese día no cuentan con almuerzo, optimizando el aprovechamiento de recursos alimentarios.

Objetivos Específicos

- Garantizar una velocidad de procesamiento síncrono que permita la gestión completa de reservas y notificaciones push masivas en una ventana crítica estricta de 30 minutos (2:00 PM a 2:30 PM).
- Proveer un mecanismo transparente y auditable de trazabilidad mediante la generación y validación de códigos QR dinámicos con tiempo de expiración controlado.
- Asegurar el cumplimiento normativo estricto con la legislación colombiana de protección de datos (Ley 1581 de 2012), restringiendo el acceso exclusivamente a la comunidad estudiantil institucional validada.

1.2. Alcance del Proyecto

El proyecto abarca la definición de requerimientos, el modelado arquitectónico, el diseño detallado de interfaces y contratos de API, la estructuración de la base de datos y la planificación del control de calidad.

Queda explícitamente fuera del alcance: el desarrollo del código fuente de las aplicaciones móvil y web; la gestión de transacciones monetarias o pasarelas de pago; y el soporte técnico físico de los dispositivos de escaneo en las instalaciones del comedor.



2. Requerimientos del Software (SRS)

2.1. Requerimientos Funcionales (RF)

ID	Nombre	Descripción	Prioridad
RF-01	Autenticación OAuth2	El sistema debe restringir el acceso permitiendo inicio de sesión únicamente a usuarios con cuentas @unipamplona.edu.co.	Alta
RF-02	Liberación Automática de Inventario	El sistema debe activar el inventario disponible exactamente a las 2:00 PM de forma síncrona, calculando la diferencia entre almuerzos preparados y entregados.	Alta
RF-03	Notificaciones Push Masivas	Al liberarse el inventario, el motor de mensajería debe enviar notificaciones push a todos los estudiantes activos en menos de 10 segundos.	Alta
RF-04	Reserva bajo Política FCFS	Las reservas se asignarán bajo First-Come, First-Served. El sistema rechazará solicitudes cuando el inventario llegue a cero.	Alta
RF-05	Generación de QR Dinámico	Al confirmar la reserva, el sistema debe generar un QR único vinculado al estudiante con vigencia máxima de 15 minutos o hasta las 2:30 PM.	Alta
RF-06	Validación por Escaneo en Comedor	El panel web del operario debe permitir el escaneo del QR, validar su estado (Activo, Expirado, Utilizado) y registrar la entrega en tiempo real.	Alta
RF-07	Tablero Analítico y de Impacto	El sistema administrativo debe consolidar reportes en tiempo real sobre raciones liberadas, efectividad de entrega y tiempos promedio.	Media

2.2. Requerimientos No Funcionales (RNF)

ID	Categoría	Descripción	Métrica
RNF-01	Rendimiento y Concurrencia	El backend debe procesar hasta 5.000 solicitudes simultáneas por segundo en el minuto de apertura (2:00 PM) sin degradación del rendimiento.	≤5ms / op



ID	Categoría	Descripción	Métrica
RNF-02	Disponibilidad Crítica	El sistema debe garantizar disponibilidad del 99.9% durante la ventana operativa de 1:55 PM a 2:40 PM.	99.9% uptime
RNF-03	Seguridad y Privacidad	Todos los datos personales deben almacenarse cifrados en reposo (AES-256) y en tránsito (TLS 1.3), con consentimiento explícito (Ley 1581/2012).	AES-256 / TLS 1.3
RNF-04	Restricción de Caducidad	Los tokens QR deben invalidarse automáticamente en el servidor al expirar los 15 minutos, impidiendo ataques de repetición.	TTL=15 min

2.3. Matriz de Trazabilidad de Requisitos

ID Requisito	Descripción	Caso de Uso	Casos de Prueba
RF-01	Autenticación con correo institucional	CU-01: Autenticar Estudiante	CP-01, CP-02
RF-02	Liberación automática 2:00 PM	CU-02: Liberar Inventario	CP-03
RF-03	Notificaciones push masivas (<10s)	CU-02: Liberar Inventario	CP-03B
RF-04	Reserva FCFS en tiempo real	CU-03: Reservar Almuerzo	CP-04, CP-05
RF-05	Generación de QR dinámico (15 min)	CU-03: Reservar Almuerzo	CP-06
RF-06	Validación por escaneo de QR	CU-04: Validar Entrega QR	CP-07, CP-08
RF-07	Tablero analítico y de impacto	CU-05: Consultar Analíticas	CP-09
RNF-01	Concurrencia 5.000 req/s	CU-03: Reservar Almuerzo	CP-LR-01 (Carga)



ID Requisito	Descripción	Caso de Uso	Casos de Prueba
RNF-02	Disponibilidad 99.9%	Todos los CU críticos	CP-HA-01 (Disponibilidad)
RNF-03	Encriptación AES-256 / TLS 1.3	CU-01: Autenticar Estudiante	CP-SEC-01
RNF-04	Caducidad automática de tokens	CU-04: Validar Entrega QR	CP-07

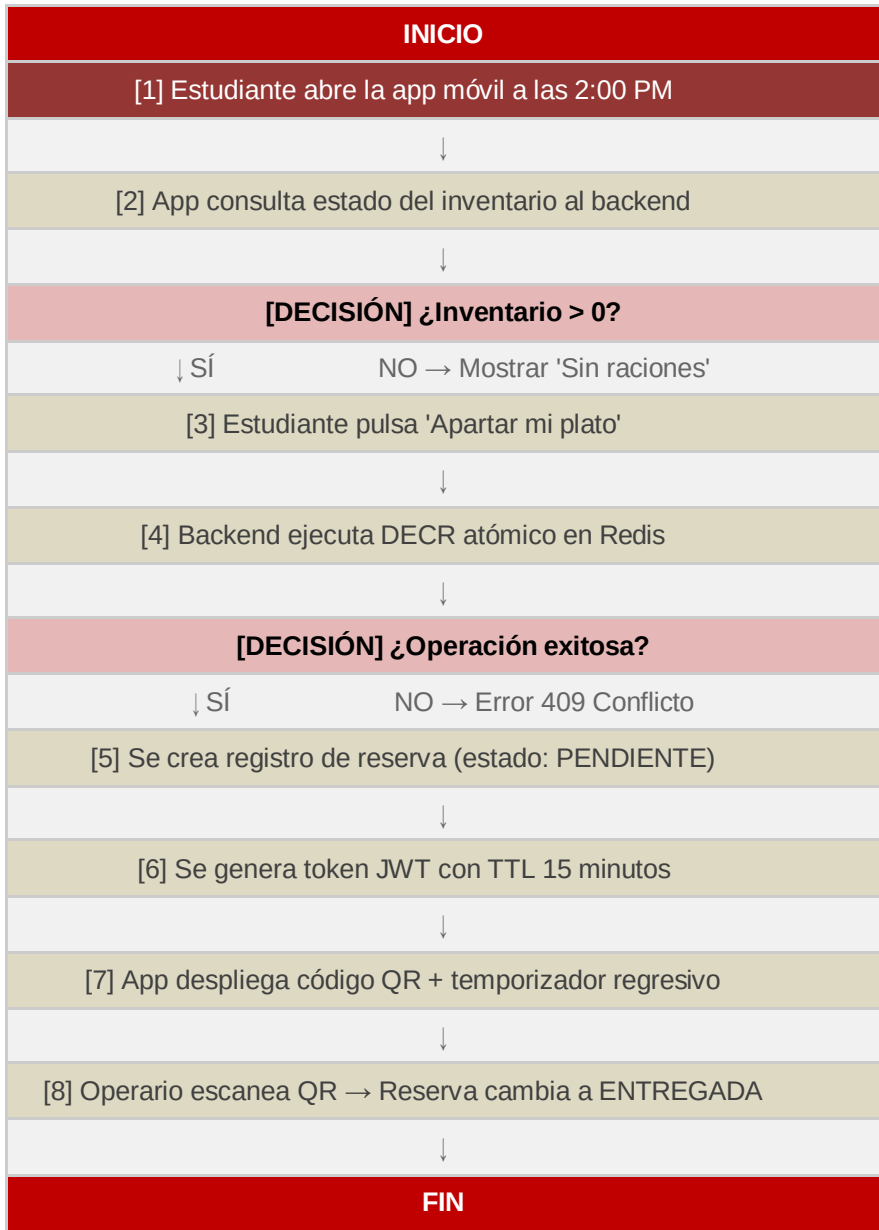
2.4. Modelos de Casos de Uso

2.4.1. Actores y Casos de Uso del Sistema

El sistema Plato Solidario cuenta con tres actores principales y cinco casos de uso nucleares:

ID	Caso de Uso	Actor	Descripción
CU-01	Autenticar Estudiante	Estudiante	Ingreso al sistema con correo @unipamplona.edu.co mediante OAuth2 institucional.
CU-02	Liberar Inventario	Sistema Administrador	Activación automática del inventario de raciones sobrantes a las 2:00 PM con notificaciones push.
CU-03	Reservar Almuerzo	Estudiante	Selección y reserva de una ración disponible bajo política FCFS con generación de QR.
CU-04	Validar Entrega QR	Operario Comedor	Escaneo y verificación del código QR del estudiante para registrar la entrega.
CU-05	Consultar Analíticas	Sistema Administrador	Visualización de métricas de impacto, raciones entregadas y tiempos de recolección.

2.4.2. Flujo de Reserva — CU-03



2.4.3. CU-03: Reservar Almuerzo (Detallado)

Precondiciones:

- El estudiante ha iniciado sesión con su cuenta institucional validada.
- El sistema se encuentra dentro de la ventana de 2:00 PM a 2:30 PM.
- El inventario disponible de raciones es mayor a cero.



Flujo Principal:

1. El estudiante recibe la notificación push o ingresa directamente a la app a las 2:00 PM.
2. La aplicación solicita el estado del inventario síncrono al backend vía API REST.
3. El estudiante hace clic en el botón "Apartar mi plato".
4. El backend procesa la transacción atómica disminuyendo en 1 el contador Redis.
5. Se crea un registro de reserva en estado PENDIENTE y un token JWT con expiración de 15 minutos.
6. La app despliega el código QR dinámico con temporizador de cuenta regresiva.

Flujo Alternativo A — Inventario Agotado:

En el paso 4, si el inventario llega a cero concurrentemente, el sistema retorna HTTP 409 Conflict y la app muestra: "Las raciones del día se han terminado".



3. Arquitectura de Software (SAD)

3.1. Diagrama de Contenedores — Modelo C4

Contenedor	Descripción y Responsabilidad
App Móvil (iOS/Android)	Interfaz nativa para el estudiante. Recibe notificaciones FCM, renderiza códigos QR a partir de tokens JWT firmados y muestra el temporizador regresivo.
Panel Web SPA (Admin/Comedor)	Single Page Application para operarios y administradores. Permite escaneo de QR mediante cámara web y visualización de analíticas en tiempo real.
API Gateway / Backend	Servicio asíncrono no bloqueante (Node.js/Fastify o Go). Centraliza seguridad, autenticación OAuth2 institucional, validación de negocio y orquestación de servicios.
Caché In-Memory (Redis)	Almacena de forma efímera y atómica el contador de raciones. Garantiza operaciones concurrentes en microsegundos mediante DECR atómico y gestiona la caducidad nativa (TTL).
Base de Datos Relacional (PostgreSQL)	Almacén persistente principal. Guarda información histórica de usuarios, raciones por fecha, auditorías de entregas y datos maestros.
Servicio de Mensajería (FCM)	Componente externo de Firebase Cloud Messaging. Recibe instrucciones del backend para el envío masivo de notificaciones push en menos de 10 segundos.

3.2. Registro de Decisiones de Arquitectura (ADR)

ADR-001: Adopción de Redis para Manejo de Inventario de Alta Concurrencia

Campo	Detalle
Estado	Aprobado — Junio 2026
Contexto	A las 2:00 PM, miles de estudiantes intentarán reservar simultáneamente un número limitado de almuerzos sobrantes (~150 raciones). El uso directo de BD relacional generaría race conditions y bloqueos de filas.
Decisión	Utilizar operaciones atómicas DECR en Redis para el control del inventario en tiempo real. La BD persistente solo se actualiza de forma asíncrona una vez confirmada la reserva en la capa in-memory.



Campo	Detalle
Consecuencias	Reducción del tiempo de respuesta de ~200ms a <5ms. Mitigación absoluta de sobreventa. Requiere lógica de conciliación al finalizar la jornada para sincronizar Redis ↔ PostgreSQL.
Alternativas Descartadas	(1) PostgreSQL con SELECT FOR UPDATE: descartado por riesgo de deadlocks. (2) Cola de mensajes (RabbitMQ/Kafka): añade latencia innecesaria para una operación que debe resolverse en milisegundos.

ADR-002: Autenticación OAuth2 con Dominio Institucional

Campo	Detalle
Estado	Aprobado — Junio 2026
Contexto	El servicio está restringido exclusivamente a la comunidad estudiantil de la Universidad de Pamplona. Se requiere validación institucional sin gestionar credenciales propias.
Decisión	Implementar OAuth2 delegando la autenticación al proveedor de identidad institucional (@unipamplona.edu.co). Los tokens de sesión son JWT firmados con clave asimétrica RS256.
Consecuencias	Se elimina la gestión de contraseñas propias. El sistema hereda la seguridad del IdP institucional. Los estudiantes sin cuenta activa quedan automáticamente excluidos.



4. Diseño del Software

4.1. Diagrama de Clases Conceptual

Clase	Atributos	Métodos	Relación
Usuario (Base)	id: UUID, nombre: String, email: String, rol: Enum	autenticar()	Base de Estudiante y Operario
Estudiante (Hereda Usuario)	codigoEstudiante: String, estadoSancion: Boolean	solicitarReserva(), cancelarReserva()	Hereda de Usuario. Tiene → Reserva (1..*)
OperarioComedor (Hereda Usuario)	idComedor: Integer	escanearCodigoQR(), registrarEntregaManual()	Hereda de Usuario. Valida → TicketQR
RacionAlimento	idRacion: UUID, fecha: Date, cantidadInicial: Integer, cantidadDisponible: Integer	liberarInventario(), decrementarCantidad()	Tiene → Reserva (1..*)
Reserva	idReserva: UUID, estudianteId: UUID, racionId: UUID, horaCreacion: Timestamp, estado: Enum	confirmarEntrega(), invalidarPorTiempo()	Pertenece a Estudiante. Genera → TicketQR (1..1)
TicketQR	tokenJWT: String, fechaExpiracion: Timestamp	generarToken(), validarFirma()	Pertenece a Reserva (1..1)

4.2. Contratos de API

Endpoint 1 — Creación de Reserva

POST /api/v1/reservas

Descripción: Crea una reserva atómica para el estudiante autenticado bajo la política FCFS.

Request Body:

```
{ "racion_id": "8f3b9c4d-2a1b-4c6d-8e7f-0a1b2c3d4e5f" }
```

Response 201 Created:

```
{ "reserva_id": "acbd1234...", "estado": "PENDIENTE", "hora_limite_recogida": "2026-06-15T14:15:00Z", "qr_token": "eyJhbGciOi..." }
```



Response 409 Conflict — Inventario Agotado:

```
{ "codigo_error": "INVENTARIO_AGOTADO", "mensaje": "No quedan raciones disponibles para reserva en la jornada actual." }
```

Endpoint 2 — Validación de Entrada QR

PATCH /api/v1/reservas/validar

Descripción: Valida el código QR presentado por el estudiante y registra la entrega de la ración.

Request Body:

```
{ "qr_token": "eyJhbGci..." }
```

Response 200 OK:

```
{ "resultado": "VALIDADO_EXITOSO", "estudiante": "Jaime Mateo Ruiz Buitrago", "hora_entrega": "2026-06-15T14:07:23Z" }
```

Response 400 — Ticket Expirado:

```
{ "resultado": "VALIDACION_FALLIDA", "codigo_error": "TICKET_EXPIRADO", "mensaje": "El QR superó el límite de vigencia de 15 minutos." }
```

Endpoint 3 — Consulta de Inventario en Tiempo Real

GET /api/v1/raciones/disponibles

Descripción: Retorna el estado actual del inventario para la jornada activa. Requiere autenticación JWT.

Response 200 OK:

```
{ "fecha": "2026-06-15", "ventana_activa": true, "raciones_disponibles": 47, "hora_cierre": "2026-06-15T14:30:00Z" }
```

4.3. Prototipos UX/UI

Pantalla	Elementos UI	Comportamiento / UX
Inicio (Estudiante)	Header con logo institucional y nombre. Indicador de estado (Cerrado / ¡Disponible!). Contador gigante de raciones. Botón CTA "Apartar mi plato" deshabilitado fuera de ventana.	Estado cerrado antes de las 2:00 PM. A las 2:00 PM: animación de activación y notificación push. Contador en tiempo real vía WebSocket. Al agotarse stock: botón desaparece.
Ticket de Reserva (QR)	Fondo verde institucional. QR central de alta densidad. Temporizador regresivo MM:SS en fuente grande. Datos del estudiante y número de ración.	Temporizador decrementa desde 15:00. A 2 minutos restantes: fondo cambia a naranja. Al llegar a 00:00: QR muestra overlay EXPIRADO. Sin captura de pantalla.
Validación Operario (Web)	Barra lateral con accesos a escaneo y analíticas. Área central con visor de cámara. Historial de entregas del día. Indicadores: verde (válido), rojo (expirado/usado).	Procesamiento automático sin botón de confirmar. Feedback inmediato: sonido + pantalla verde o roja. Registro automático de hora de entrega.



Pantalla	Elementos UI	Comportamiento / UX
Tablero Analítico (Admin)	Tarjetas KPI: raciones liberadas, entregadas, expiradas. Gráfico de barras por franja horaria. Mapa de calor por semana. Exportación CSV/Excel.	Datos actualizados en tiempo real durante la ventana. Resumen diario desde las 3:00 PM. Filtros por fecha, semana y programa académico.



5. Construcción de Software

5.1. Script de Base de Datos (DDL)

Script de creación de la estructura relacional del sistema Plato Solidario en PostgreSQL 15+:

```
CREATE TYPE rol_usuario AS ENUM ('ESTUDIANTE', 'OPERARIO', 'ADMINISTRADOR');
CREATE TYPE estado_reserva AS ENUM ('PENDIENTE', 'ENTREGADA', 'EXPIRADA', 'CANCELADA');

CREATE TABLE usuarios (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  nombre_completo VARCHAR(150) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL CHECK (email LIKE '%@unipamplona.edu.co'),
  rol rol_usuario NOT NULL DEFAULT 'ESTUDIANTE',
  fecha_registro TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  activo BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE raciones_diarias (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  fecha DATE UNIQUE NOT NULL DEFAULT CURRENT_DATE,
  cantidad_inicial INT NOT NULL CHECK (cantidad_inicial >= 0),
  cantidad_disponible INT NOT NULL CHECK (cantidad_disponible >= 0),
  hora_liberacion TIMESTAMP WITH TIME ZONE,
  hora_cierre TIMESTAMP WITH TIME ZONE
);

CREATE TABLE reservas (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  usuario_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE RESTRICT,
  racion_id UUID NOT NULL REFERENCES raciones_diarias(id) ON DELETE RESTRICT,
  hora_reserva TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  hora_limite TIMESTAMP WITH TIME ZONE NOT NULL,
  estado estado_reserva NOT NULL DEFAULT 'PENDIENTE',
  token_verificacion TEXT UNIQUE NOT NULL
);

CREATE INDEX idx_reservas_usuario_fecha ON reservas(usuario_id, hora_reserva);
CREATE INDEX idx_reservas_estado ON reservas(estado) WHERE estado = 'PENDIENTE';
CREATE INDEX idx_raciones_fecha ON raciones_diarias(fecha);
```



5.2. Diccionario de Datos

Tabla: usuarios

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	NO	PK	Identificador único autogenerado para cada usuario.
nombre_completo	VARCHAR(150)	NO	—	Nombre legal completo del usuario.
email	VARCHAR(100)	NO	UK	Correo institucional. Debe terminar en @unipamplona.edu.co.
rol	ENUM	NO	—	Rol del usuario: ESTUDIANTE, OPERARIO o ADMINISTRADOR.
fecha_registro	TIMESTAMP TZ	NO	—	Marca de tiempo de creación del registro.
activo	BOOLEAN	NO	—	Indica si la cuenta está habilitada para operar.

Tabla: raciones_diarias

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	NO	PK	Identificador único de la tanda de raciones de un día.
fecha	DATE	NO	UK	Fecha de la jornada. UNIQUE: una sola tanda por día.
cantidad_inicial	INTEGER	NO	—	Total de raciones preparadas por el comedor al inicio del día.
cantidad_disponible	INTEGER	NO	—	Raciones sobrantes no reclamadas, disponibles para redistribución.
hora_liberacion	TIMESTAMP TZ	SÍ	—	Momento exacto en que el inventario fue activado (~2:00 PM).



Columna	Tipo	Nulo	Clave	Descripción
hora_cierre	TIMESTAMP TZ	Sí	—	Momento en que se cerró la ventana de reservas (~2:30 PM).

Tabla: reservas

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	NO	PK	Identificador único autogenerado para cada reserva.
usuario_id	UUID	NO	FK	Relación con el estudiante solicitante (ON DELETE RESTRICT).
racon_id	UUID	NO	FK	Vínculo a la tanda de raciones del día (ON DELETE RESTRICT).
hora_reserva	TIMESTAMP TZ	NO	—	Marca de tiempo exacta de asignación de la reserva.
hora_limite	TIMESTAMP TZ	NO	—	Expiración calculada como hora_reserva + 15 min (no mayor a 14:30).
estado	ENUM	NO	—	Ciclo de vida: PENDIENTE, ENTREGADA, EXPIRADA, CANCELADA.
token_verificacion	TEXT	NO	UK	Token JWT RS256. Restricción UNIQUE: un token por reserva.

5.3. Guía Técnica del Desarrollador

Requisitos del Entorno:

Herramienta	Versión Mínima	Propósito
Docker Desktop	v24+	Orquestación de servicios locales (Redis, PostgreSQL, API).
Node.js LTS	v20+	Runtime del servidor API (Fastify) y herramientas de build.



Herramienta	Versión Mínima	Propósito
PostgreSQL	v15+	Base de datos relacional principal del sistema.
Redis Server	v7.2+	Caché in-memory para control de inventario concurrente.
Git	v2.40+	Control de versiones del repositorio de código fuente.

Estructura Base del Repositorio Backend:

```
/plato-solidario-api
├── /src
│   ├── /config           # Configuraciones: Redis, DB, Seguridad JWT
│   ├── /controllers      # Controladores HTTP: Auth, Reservas, Analíticas
│   ├── /domain           # Modelos de dominio puro y lógica de negocio
│   ├── /jobs             # Tareas cron (expiración automática 2:30 PM)
│   ├── /repositories     # Capa de abstracción de acceso a datos
│   ├── /middleware       # Auth middleware, rate limiting, validación
│   └── app.js            # Punto de entrada de la aplicación
├── /tests
│   ├── /unit             # Pruebas unitarias (Jest)
│   ├── /integration      # Pruebas de integración (Supertest + BD efímera)
│   └── /load              # Scripts de carga (K6 / Apache JMeter)
├── docker-compose.yml
├── .env.example
└── package.json
```



6. Pruebas de Software

6.1. Plan de Pruebas

Nivel	Tipo de Prueba	Descripción
Nivel 1	Pruebas Unitarias Aisladas	Cobertura exhaustiva sobre la lógica de generación y firma del token JWT, validación de zonas horarias locales y cálculo de expiración de QR.
Nivel 2	Pruebas de Integración	Validación de la persistencia correcta y el decremento exacto en Redis al simular solicitudes simultáneas bajo FCFS. Incluye pruebas de sincronización Redis ↔ PostgreSQL.
Nivel 3	Pruebas de Carga y Estrés Crítico	Ejecución de scripts de carga con K6 o Apache JMeter para replicar la entrada concurrente masiva de 5.000 usuarios exactamente a las 2:00:00 PM, midiendo tasas de error y tiempos de respuesta.

Criterios de Aceptación:

Criterio	Condición de Aceptación
Cobertura mínima	No se aprobará ningún PR si la cobertura total cae por debajo del 85%.
Tasa de errores en carga	La tasa de error bajo 5.000 usuarios concurrentes no debe superar el 0.1%.
Tiempo de respuesta P95	El P95 del endpoint POST /reservas debe ser < 50ms.
Integridad de stock	Bajo ninguna circunstancia el sistema puede entregar más raciones de las disponibles (inventario nunca negativo).
Caducidad de tokens	El 100% de los tokens QR deben invalidarse correctamente al alcanzar su TTL de 15 minutos.
Autenticación	El 100% de los accesos con correo no institucional deben ser rechazados con HTTP 403.



6.2. Casos de Prueba Detallados

ID	Componente	Escenario	Datos de Entrada	Resultado Esperado
CP-01	Módulo Auth	Autenticación exitosa con correo institucional válido.	j.mateo@unipamplona.edu.co	HTTP 200 OK. Emisión de JWT de sesión válido.
CP-02	Módulo Auth	Bloqueo de dominios comerciales externos.	usuario@gmail.com	HTTP 403 Forbidden. Sin emisión de token.
CP-03	Motor Inventario	Liberación automática a las 2:00 PM exactas.	Hora del sistema: 14:00:00	Evento disparado. hora_liberacion registrada en BD.
CP-03B	Motor Notif.	Envío masivo de push en < 10 segundos.	1.000 dispositivos suscritos activos.	100% dispositivos reciben notificación en ≤10s.
CP-04	Motor Reservas	Reserva exitosa con stock positivo disponible.	Stock Redis = 10. racion_id válido.	HTTP 201. Stock pasa a 9. Reserva PENDIENTE en BD.
CP-05	Motor Reservas	Control de condición de carrera (conurrencia).	Stock Redis = 1. Dos peticiones simultáneas.	Petición A: HTTP 201 (Stock=0). Petición B: HTTP 409. Stock nunca llega a -1.
CP-06	Módulo QR	Generación de QR con expiración correcta.	Reserva confirmada a las 14:02:15.	Token JWT con exp=14:17:15. QR renderizado con temporizador activo.
CP-07	Módulo QR Scan	Validación de QR expirado.	Escaneo a 2:16 PM de ticket creado a 2:00 PM.	HTTP 400. TICKET_EXPIRADO. Estado → EXPIRADA en BD.
CP-08	Módulo QR Scan	Intento de reutilización de QR ya utilizado.	Token de reserva con estado ENTREGADA.	HTTP 400. TICKET_YA_UTILIZADO. Sin modificación.
CP-09	Tablero Admin	Visualización de analíticas del día en tiempo real.	Fecha con ≥10 reservas en estados mixtos.	Dashboard muestra KPIs actualizados: liberadas,



ID	Componente	Escenario	Datos de Entrada	Resultado Esperado
				entregadas, expiradas y tasa de éxito.
CP-LR-01	Carga (K6)	Estrés con 5.000 usuarios concurrentes a 2:00 PM.	5.000 requests. Stock inicial = 150.	Exactamente 150 HTTP 201. 4.850 HTTP 409. Error < 0.1%. P95 < 50ms.
CP-HA-01	Disponibilidad	Sistema disponible durante la ventana crítica completa.	Monitoreo 1:55 PM a 2:40 PM.	Uptime 100% durante la ventana. Sin interrupciones.
CP-SEC-01	Seguridad	Datos en tránsito protegidos con TLS 1.3.	Inspección del handshake SSL.	Conexión establece TLS 1.3. Datos ilegibles en texto plano. Certificado válido.

6.3. Reporte de Cobertura

Capa del Sistema	Cobertura Líneas	Cobertura Ramas	Herramienta
Controladores y Enrutamiento (API Gateway)	≥ 80%	≥ 75%	Jest + Supertest
Servicios de Negocio (Lógica FCFS y QR Core)	≥ 95%	≥ 95%	Jest (Branch Coverage)
Acceso a Datos e Infraestructura	≥ 85%	≥ 80%	Jest + BD efímera Docker
Módulo de Autenticación OAuth2	≥ 90%	≥ 90%	Jest + mock IdP
Jobs Cron (expiración automática)	≥ 85%	≥ 85%	Jest con fake timers



7. Operaciones de Ingeniería del Software

7.1. Scripts de Infraestructura como Código (IaC)

La infraestructura de Plato Solidario se gestiona mediante Terraform, garantizando reproducibilidad, versionamiento y despliegues auditables en entornos cloud.

Recurso	Proveedor	Configuración Clave
Instancia API Backend	AWS EC2 / GCP Compute	t3.medium, auto-scaling 2–10 instancias
Base de datos PostgreSQL	AWS RDS	db.t3.small, Multi-AZ, backups diarios 7 días
Caché Redis	AWS ElastiCache	cache.t3.micro, cluster mode disabled, TTL nativo
Balanceador de carga	AWS ALB	HTTPS:443, certificado ACM, health checks /health
Almacenamiento objetos	AWS S3	Bucket privado para logs de auditoría y exportaciones
CDN	AWS CloudFront	Distribución de la SPA del panel web administrador

7.2. Pipelines de CI/CD

Pipeline: rama develop → entorno dev

Etapas	Herramienta	Condición de Éxito
1. Lint y formato	ESLint + Prettier	0 errores de estilo
2. Pruebas unitarias	Jest	Cobertura ≥ 85% global, ≥ 95% dominio
3. Análisis estático	SonarQube	Quality Gate: 0 bugs críticos
4. Build de imagen	Docker	Imagen construida y etiquetada con SHA del commit



Etapa	Herramienta	Condición de Éxito
5. Despliegue dev	Terraform apply	Entorno dev actualizado sin errores
6. Pruebas de humo	curl / Postman CLI	Endpoints /health y /api/v1/raciones responden 200

Pipeline: rama main → entorno producción

Etapa	Herramienta	Condición de Éxito
1–3. (Igual que develop)	ESLint / Jest / SonarQube	Mismos umbrales
4. Pruebas de integración	Supertest + BD Docker efímera	100% casos de integración pasan
5. Pruebas de carga	K6	P95 < 50ms, error rate < 0.1% a 5.000 req/s
6. Aprobación manual	GitHub Environments	Revisión obligatoria de 1 maintainer
7. Despliegue prod	Terraform apply (prod.tfvars)	Rolling update sin downtime (blue/green)
8. Verificación post-deploy	Datadog Synthetic	Monitor activo confirma disponibilidad

7.3. Tableros de Monitoreo

Métrica Monitorizada	Herramienta	Umbral de Alerta
Latencia P95 endpoint POST /reservas	Datadog APM	> 50ms → WARNING; > 100ms → CRITICAL
Tasa de errores HTTP 5xx	Datadog Logs	> 0.1% → CRITICAL (PagerDuty)



Métrica Monitorizada	Herramienta	Umbral de Alerta
Disponibilidad API (uptime)	Datadog Synthetics	< 99.9% en ventana → CRITICAL
Contador Redis (raciones disponibles)	Datadog Redis Integration	Valor negativo → CRITICAL
Conexiones activas PostgreSQL	Datadog DB Monitoring	> 80% del pool → WARNING
Uso de CPU instancias backend	Datadog Infrastructure	> 80% → scale-out trigger
Latencia FCM (notificaciones push)	Datadog Custom Metrics	> 10s envío masivo → CRITICAL
Certificado TLS vigencia	Datadog SSL Monitor	< 30 días → WARNING



8. Mantenimiento de Software

8.1. Manual de Operaciones

Código / Síntoma	Diagnóstico	Acción Correctiva
API responde HTTP 503	Backend sin instancias sanas en el ALB	1. Verificar logs en CloudWatch. 2. Revisar health checks del ALB. 3. Reiniciar instancia EC2 con menor uptime.
Redis devuelve valor negativo en raciones	Race condition no controlada o fallo en DECR	1. Ejecutar: redis-cli SET raciones_YYYY-MM-DD 0. 2. Conciliar contra tabla reservas en PostgreSQL. 3. Notificar al equipo de desarrollo.
FCM no entrega notificaciones	Token Firebase expirado o cuota excedida	1. Renovar service account key en Firebase Console. 2. Actualizar secret en AWS Secrets Manager. 3. Reiniciar servicio de mensajería.
QR marcado EXPIRADO antes de 15 min	Desincronización de relojes entre cliente y servidor	1. Verificar NTP: sudo systemctl status chronyd. 2. Sincronizar: sudo chronyc makestep. 3. Ajustar TTL con margen de 30s en la config.
PostgreSQL: Too many connections	Pool de conexiones agotado bajo alta carga	1. Aumentar max_connections en RDS. 2. Revisar fugas de conexión. 3. Escalar instancia RDS verticalmente.
Pipeline CI/CD falla en etapa de carga	K6 supera umbral P95 o error rate	1. Revisar métricas Datadog del entorno staging. 2. Identificar endpoint con mayor latencia. 3. Escalar instancias o revisar query N+1 en ORM.

8.2. Plan de Parches y Actualizaciones

Ciclo	Tipo de Actualización	Proceso	Responsable
Semanal	Parches de seguridad críticos (CVE Alto/Crítico)	1. Alerta automática Dependabot. 2. PR automático. 3. Review + merge en 48h.	DevSecOps
Mensual	Actualizaciones menores de dependencias npm	1. npm audit fix. 2. Pruebas regresión completas. 3. Despliegue en staging 48h antes de prod.	Dev Lead
Trimestral	Actualizaciones de runtime (Node.js LTS, PostgreSQL)	1. Rama feature/upgrade-X. 2. Pipeline completo. 3. Aprobación manual en prod.	Arquitecto



Ciclo	Tipo de Actualización	Proceso	Responsable
Semestral	Revisión de imagen base Docker y SO	1. Rebuild completo de imagen. 2. Escaneo Trivy. 3. Pruebas de regresión end-to-end.	DevOps

8.3. Registro de Deuda Técnica

ID	Descripción	Impacto	Esfuerzo	Prioridad
DT-01	Conciliación Redis↔PostgreSQL al cierre de jornada es proceso manual.	Riesgo de inconsistencia de datos si falla el job cron.	M (3 días)	Alta
DT-02	Falta paginación en endpoint GET /api/v1/reportes/diario.	Degradación de rendimiento con > 6 meses de datos históricos.	S (1 día)	Media
DT-03	Mock del IdP institucional en pruebas unitarias es frágil.	Falsos positivos en CI si cambia la estructura del token OAuth2.	M (2 días)	Media
DT-04	Sin circuit breaker para llamadas al servicio FCM externo.	Cascada de errores si FCM presenta latencia alta.	L (5 días)	Alta
DT-05	Logs de auditoría no estructurados (texto plano vs JSON).	Dificultad para consultas y alertas en Datadog.	S (1 día)	Baja
DT-06	Panel web no soporta modo oscuro.	Experiencia degradada para operarios nocturnos.	M (3 días)	Baja
DT-07	Método demasiado largo (>30 líneas) en reservasController.js:87 (CS-01).	Dificultad de mantenimiento y pruebas unitarias del controlador.	S (1 día)	Media



9. Gestión de Configuración del Software

9.1. Estrategia de Ramas Git (Gitflow Adaptado)

Rama	Propósito	Reglas de Protección
main	Código en producción. Siempre estable y desplegable.	Protegida. Requiere 1 aprobación + pipeline verde. Sin push directo.
develop	Integración continua de features. Base para staging.	Protegida. Requiere pipeline verde. Sin push directo.
feature/xxx	Desarrollo de nuevas funcionalidades o mejoras.	Rama de vida corta. Se elimina al hacer merge a develop.
hotfix/xxx	Correcciones urgentes directamente a producción.	Se crea desde main. Se fusiona a main Y develop. Requiere aprobación.
release/vX.Y.Z	Preparación de una versión para producción.	Solo correcciones de bugs. Se fusiona a main con tag semántico.

9.2. Notas de Liberación (Release Notes)

Versión	Fecha	Cambios Incluidos	Tipo
v0.1.0	Abr 2026	Autenticación OAuth2 institucional (RF-01). Estructura base de BD PostgreSQL.	Alpha
v0.2.0	May 2026	Motor de liberación de inventario Redis (RF-02). Notificaciones FCM (RF-03).	Beta
v0.3.0	May 2026	Reservas FCFS con control de concurrencia (RF-04). Generación QR JWT (RF-05).	Beta
v0.4.0	Jun 2026	Panel operario: validación QR (RF-06). Tablero analítico básico (RF-07).	Beta
v1.0.0	Jun 2026	Release de producción. Pruebas de carga K6 superadas. Infraestructura IaC completa.	Producción



9.3. Inventario de Artefactos y Dependencias

Paquete	Versión	Propósito	Licencia
fastify	^4.26.0	Framework HTTP no bloqueante del backend API.	MIT
@fastify/jwt	^8.0.0	Firma y verificación de tokens JWT RS256.	MIT
ioredis	^5.3.2	Cliente Redis para operaciones DECR atómicas.	MIT
pg (node-postgres)	^8.11.3	Driver PostgreSQL para la capa de persistencia.	MIT
firebase-admin	^12.0.0	SDK para envío masivo de notificaciones FCM.	Apache 2.0
qrcode	^1.5.3	Generación de imágenes QR a partir del token JWT.	MIT
zod	^3.22.4	Validación de esquemas de entrada en los endpoints.	MIT
jest	^29.7.0	Framework de pruebas unitarias e integración.	MIT
k6 (binary)	v0.50.0	Herramienta de pruebas de carga y estrés.	AGPL-3.0
terraform	v1.7.x	Infraestructura como código para despliegue cloud.	BUSL-1.1



10. Gestión de Ingeniería del Software

10.1. WBS — Estructura Desglosada del Trabajo

Código	Entregable / Actividad	Responsable	Duración Est.
1.0	PLATO SOLIDARIO — PROYECTO COMPLETO	—	16 semanas
1.1	Gestión del Proyecto	Director	16 sem (paralelo)
1.2	Requerimientos (SRS)	Analista	2 semanas
1.3	Arquitectura (SAD)	Arquitecto	2 semanas
1.4	Diseño Detallado	Dev Lead	2 semanas
1.5	Construcción	Equipo Dev	6 semanas
1.5.1	DDL y scripts de base de datos	Backend Dev	3 días
1.5.2	Módulo autenticación OAuth2	Backend Dev	1 semana
1.5.3	Motor reservas FCFS + Redis	Backend Dev	2 semanas
1.5.4	Servicio QR y notificaciones FCM	Backend Dev	1 semana
1.5.5	App móvil (Flutter)	Mobile Dev	4 semanas
1.5.6	Panel web SPA (React.js)	Frontend Dev	3 semanas
1.6	Pruebas	QA Engineer	3 semanas
1.7	Despliegue e Infraestructura	DevOps	1 semana



10.2. Cronograma del Proyecto

Semana	Hito	Entregable Verificable	Estado
S1–S2	Kickoff y Requerimientos	Documento SRS aprobado por stakeholders	Completado
S3–S4	Arquitectura y ADRs	SAD v1.0 + Contratos API OpenAPI	Completado
S5–S6	Diseño detallado	Diagrama de clases, prototipos UX aprobados	Completado
S7–S8	Construcción — Auth y BD	Módulo OAuth2 funcional + DDL migrado	Completado
S9–S11	Construcción — Motor FCFS	Reservas funcionando con Redis en dev	Completado
S12–S13	Construcción — QR y FCM	QR dinámico + notificaciones push activas	Completado
S14	Pruebas completas	Reporte QA: cobertura ≥ 85%, carga K6 OK	Completado
S15	Despliegue staging	Entorno staging validado por stakeholders	Completado
S16	Release v1.0.0 producción	Sistema en vivo + documentación entregada	Completado

10.3. Matriz de Riesgos del Proyecto

ID	Riesgo	Prob.	Impacto	Prior.	Plan de Mitigación
R-01	Caída de Redis durante ventana 2:00–2:30 PM	Baja (2)	Crítico (5)	10	Réplica Redis en standby. Fallback a BD con pessimistic locking.
R-02	IdP institucional no disponible	Media (3)	Alto (4)	12	Caché local de tokens válidos por 4 horas. Comunicado a TI universitaria.
R-03	FCM supera cuota de mensajes gratuitos	Media (3)	Medio (3)	9	Monitorear cuota mensual. Suscripción plan pagado como contingencia.



ID	Riesgo	Prob.	Impacto	Prior.	Plan de Mitigación
R-04	Sobrecarga de la BD PostgreSQL en pico	Baja (2)	Alto (4)	8	Connection pooling (PgBouncer). Redis absorbe la carga crítica de inventario.
R-05	Cambio de estructura del token OAuth2 institucional	Baja (2)	Medio (3)	6	Prueba de contrato con IdP (contract testing). Alertas automáticas en CI.
R-06	Dispositivo de escaneo QR falla en comedor	Media (3)	Medio (3)	9	Panel web como alternativa de escaneo. Proceso manual de respaldo documentado.
R-07	Vulnerabilidad de seguridad en dependencia npm	Alta (4)	Alto (4)	16	Dependabot activo. Política de parches críticos en 48h (ver sección 9.2).



11. Modelos y Métodos de Ingeniería del Software

11.1. Historias de Usuario

ID	Historia de Usuario	Criterios de Aceptación	Prioridad
HU-01	Como estudiante, quiero recibir una notificación push a las 2:00 PM cuando haya raciones disponibles, para poder reservar mi almuerzo a tiempo.	1. Notificación llega en ≤10s. 2. Incluye cantidad disponible. 3. Tap abre la app en pantalla de reserva.	Alta
HU-02	Como estudiante, quiero ver un contador en tiempo real de raciones disponibles en la app, para saber si vale la pena intentar reservar.	1. Contador vía WebSocket. 2. Al llegar a 0 muestra "Agotado". 3. Botón se deshabilita automáticamente.	Alta
HU-03	Como estudiante, quiero reservar mi ración en un solo toque y recibir inmediatamente un código QR, para ir al comedor sin demoras.	1. Respuesta en ≤5ms. 2. QR visible con temporizador 15 min. 3. Confirmación con número de ración asignado.	Alta
HU-04	Como estudiante, quiero ver una alerta visual cuando mi QR esté por expirar, para llegar al comedor a tiempo.	1. Fondo cambia a naranja con 2 min restantes. 2. Sonido/vibración de alerta. 3. Al expirar: overlay EXPIRADO.	Media
HU-05	Como operario del comedor, quiero escanear el QR del estudiante con la cámara y ver la confirmación inmediatamente, para agilizar la cola.	1. Detección automática sin botón confirmar. 2. Pantalla verde (éxito) o roja (error) en < 1s. 3. Registro automático de hora de entrega.	Alta
HU-06	Como administrador, quiero ver un tablero con métricas diarias de raciones entregadas y desperdicio, para tomar decisiones de planificación.	1. Dashboard disponible desde las 3:00 PM. 2. KPIs: liberadas, entregadas, expiradas, tasa éxito. 3. Exportación a CSV/Excel.	Media
HU-07	Como administrador, quiero que el sistema rechace automáticamente accesos de correos no institucionales, para proteger la comunidad.	1. Correos no @unipamplona.edu.co reciben HTTP 403. 2. Intento queda en log de auditoría. 3. Sin emisión de ningún token.	Alta



11.2. Diagrama de Dominio (DDD)

Bounded Context 1: Gestión de Inventario y Reservas

Elemento DDD	Nombre	Responsabilidad
Aggregate Root	Reserva	Controla el ciclo de vida completo del ticket: PENDIENTE → ENTREGADA / EXPIRADA / CANCELADA.
Entity	Estudiante	Identidad persistente. Valida dominio institucional. Mantiene historial de reservas.
Entity	RacionDiaria	Representa el inventario de un día. Coordina la liberación y el decremento atómico.
Value Object	TicketQR	Inmutable. Encapsula el token JWT, la expiración y el estado de validez.
Domain Event	InventarioLiberado	Disparado a las 2:00 PM. Activa las notificaciones FCM masivas.
Domain Event	ReservaConfirmada	Emitido al crear la reserva. Persiste en PostgreSQL de forma asíncrona.
Domain Service	GestorFCFS	Coordina el DECR atómico en Redis y garantiza invariante: inventario ≥ 0.

Bounded Context 2: Autenticación y Control de Acceso

Elemento DDD	Nombre	Responsabilidad
Aggregate Root	SesionUsuario	Gestiona el ciclo de vida del token de sesión JWT institucional.
Entity	IdentidadInstitucional	Representa la cuenta @unipamplona.edu.co validada por el IdP externo.
Value Object	TokenSesion	Inmutable. Contiene claims JWT: sub, rol, exp, iat firmados con RS256.
Domain Service	ValidadorDominio	Verifica que el email pertenezca al dominio institucional antes de emitir token.



Elemento DDD	Nombre	Responsabilidad
Anti-Corruption Layer	OAuth2Adapter	Traduce la respuesta del IdP institucional al modelo de dominio interno.



12. Proceso de Ingeniería del Software

12.1. Definición de Listo (DoR) y Hecho (DoD)

Definición de Listo (DoR — Definition of Ready):

#	Criterio DoR	Verificado por
1	La historia tiene título, descripción en formato "Como / Quiero / Para" y criterios de aceptación redactados.	Product Owner
2	La historia está estimada en puntos de historia por el equipo (Planning Poker).	Equipo Dev
3	No tiene dependencias bloqueantes sin resolver.	Scrum Master
4	Los prototipos o mockups relevantes están aprobados por el PO.	Product Owner
5	La historia es alcanzable dentro de un sprint (≤ 8 puntos).	Dev Lead
6	Los criterios de aceptación son verificables y no ambiguos.	QA Engineer

Definición de Hecho (DoD — Definition of Done):

#	Criterio DoD	Verificado por
1	El código está implementado, revisado mediante Pull Request y aprobado por al menos 1 desarrollador senior.	Dev Lead
2	Las pruebas unitarias cubren $\geq 85\%$ de las líneas del código nuevo; $\geq 95\%$ para lógica de dominio (FCFS, QR).	QA / CI
3	El pipeline de CI/CD pasa completo en la rama develop (lint, tests, SonarQube Quality Gate).	DevOps / CI
4	La funcionalidad fue probada en el entorno de staging y validada contra los criterios de aceptación.	QA Engineer
5	La documentación técnica relevante (contratos API, README) fue actualizada si corresponde.	Dev responsable



#	Criterio DoD	Verificado por
6	No se introdujeron vulnerabilidades de seguridad nuevas (reporte Dependabot limpio).	DevSecOps
7	El Product Owner aceptó formalmente la historia en la sesión de revisión de sprint.	Product Owner

12.2. Retrospectiva — Sprint Final (Start / Stop / Continue)

Categoría	Observación	Acción de Mejora
Continuar	El uso de Redis para control de inventario eliminó completamente las condiciones de carrera identificadas en el spike técnico inicial.	Mantener este patrón en futuros módulos con alta concurrencia.
Continuar	Las pruebas de carga con K6 antes del release evitaban sorpresas en producción durante la ventana crítica.	Incluir K6 como gate obligatorio en el pipeline de producción.
Continuar	Los ADRs documentados facilitaron la toma de decisiones sin reuniones adicionales.	Exigir ADR para toda decisión arquitectónica en futuros proyectos.
Dejar de hacer	Acumular tareas de documentación para el final del sprint genera estrés y documentación incompleta.	Incorporar la documentación como criterio del DoD desde el primer sprint.
Dejar de hacer	Las reuniones de sincronización de más de 30 minutos reducen el tiempo efectivo de desarrollo.	Fijar límite de 20 minutos para dailies con agenda previa obligatoria.
Comenzar	No se realizaron pruebas de usabilidad con estudiantes reales antes del lanzamiento.	Planificar sesión de UX Research con 5 usuarios representativos en la siguiente iteración.
Comenzar	Faltó un ambiente de preproducción con datos realistas para las pruebas de carga.	Crear dataset de prueba de 10.000 usuarios ficticios para próximos sprints de carga.



13. Calidad del Software

13.1. Reporte de Análisis Estático de Código (SonarQube)

Métrica SonarQube	Resultado Obtenido	Umbral Mínimo	Estado
Bugs críticos	0	0	Aprobado
Vulnerabilidades de seguridad	0	0	Aprobado
Code Smells (total)	14	≤ 30	Aprobado
Deuda técnica estimada	2h 40min	≤ 8h	Aprobado
Duplicación de código	1.8%	≤ 3%	Aprobado
Cobertura de pruebas (global)	87.3%	≥ 85%	Aprobado
Cobertura módulo dominio	96.1%	≥ 95%	Aprobado
Quality Gate general	PASSED	PASSED	Aprobado

Code Smells identificados (extracto):

ID	Tipo	Localización	Severidad	Acción
CS-01	Método demasiado largo (> 30 líneas)	reservasController.js:87	Minor	DT-07: refactorizar
CS-02	Magic number sin constante nombrada	qrService.js:43 (900000)	Minor	Extraer TTL_MS constant
CS-03	Variable no utilizada	analitcsRepository.js:12	Info	Eliminar en PR-041



13.2. Métricas de Calidad del Producto

Métrica	Valor Medido	Referencia / Benchmark	Interpretación
Densidad de defectos	0.8 bugs / KLOC	Industria: 1–25 bugs/KLOC (Capers Jones)	Excelente — código de alta calidad
Complejidad ciclomática promedio	4.2 por función	Recomendado: ≤ 10 (McCabe)	Bajo riesgo — fácil de mantener y probar
Complejidad ciclomática máxima	11 (fcfsService.js)	Límite crítico: > 20	Aceptable — candidato a refactoring futuro (DT-01)
Índice de mantenibilidad (MI)	82 / 100	Bueno: 65–85; Excelente: > 85	Bueno — código legible y estructurado
Líneas de código (LOC)	3.847 LOC	Proyecto pequeño: < 10 KLOC	Tamaño apropiado para el alcance definido
Ratio prueba/código	1.4:1 (test LOC vs src LOC)	Recomendado: ≥ 1:1	Cobertura de pruebas sólida
Acoplamiento eferente promedio	3.1	Recomendado: ≤ 5	Bajo acoplamiento entre módulos



14. Seguridad del Software

14.1. Modelo de Amenazas (STRIDE)

ID	Categoría STRIDE	Amenaza Identificada	Componente Afectado	Control Implementado
AM-01	Spoofing (Suplantación)	Actor malicioso intenta autenticarse con credenciales robadas o falsificadas.	Módulo OAuth2	OAuth2 delegado al IdP institucional. JWT RS256 con firma asimétrica. Validación de dominio @unipamplona.edu.co.
AM-02	Tampering (Manipulación)	Modificación del payload del token JWT para alterar el rol o el ID del estudiante.	TicketQR / API Gateway	Verificación de firma JWT en cada request. Clave pública RS256 almacenada en servidor. HTTP 401 ante firma inválida.
AM-03	Repudiation (Repudio)	Un estudiante niega haber realizado una reserva o el operario niega haber validado un QR.	Tabla reservas / Logs	Log inmutable de auditoría en S3 (append-only). Registros con timestamp, IP y user-agent. Correlación por reserva_id.
AM-04	Information Disclosure (Divulgación)	Exposición de datos personales de estudiantes en tránsito o en reposo.	API / PostgreSQL	TLS 1.3 obligatorio en todos los endpoints. Cifrado AES-256 en reposo (RDS). Cumplimiento Ley 1581/2012.
AM-05	Denial of Service (DoS)	Flood de requests al endpoint POST /reservas para colapsar el servicio en la ventana crítica.	API Gateway	Rate limiting por IP (100 req/min). AWS WAF en el ALB. Redis absorbe la carga de inventario, aislando la BD.
AM-06	Elevation of Privilege (Escalada)	Un estudiante intenta acceder a endpoints de administrador o de operario.	Middleware de roles	Middleware de autorización RBAC basado en claim "rol" del JWT. Guards diferenciados por ruta: /admin, /operario, /estudiante.
AM-07	Replay Attack (Reutilización QR)	Un actor captura un QR válido y lo presenta nuevamente para obtener una segunda ración.	TicketQR / PATCH	Estado de reserva verificado antes de cada validación. Una vez ENTREGADA, HTTP 400 TICKET_YA_UTILIZADO. TTL de 15 min en Redis.



14.2. Reporte de Análisis SAST y DAST

Análisis Estático de Seguridad (SAST) — Semgrep CE:

Categoría OWASP	Hallazgos	Severidad Máx.	Estado
A01 — Broken Access Control	0 hallazgos	—	Sin vulnerabilidades
A02 — Cryptographic Failures	0 hallazgos	—	Sin vulnerabilidades
A03 — Injection (SQL / NoSQL)	0 hallazgos	—	Queries parametrizados
A05 — Security Misconfiguration	1 hallazgo (resuelto)	Medium	Corregido en PR-038
A07 — Auth Failures	0 hallazgos	—	Sin vulnerabilidades
A09 — Logging Failures	1 hallazgo (resuelto)	Low	Corregido en PR-039
Total hallazgos activos	0	—	SAST aprobado

Análisis Dinámico de Seguridad (DAST) — OWASP ZAP:

Prueba DAST	Resultado	Estado
Inyección SQL en parámetros de entrada	Sin vulnerabilidades — queries parametrizados con pg	Pasó
Cross-Site Scripting (XSS) en panel web	Sin vulnerabilidades — React escapa output automáticamente	Pasó
Headers de seguridad HTTP (CSP, HSTS, X-Frame)	Todos los headers configurados correctamente	Pasó
Fuerza bruta en endpoint de autenticación	Bloqueado a los 5 intentos — rate limiter activo	Pasó
Exposición de tokens en logs	Sin tokens en logs — enmascarados con asteriscos	Pasó



Prueba DAST	Resultado	Estado
Acceso a rutas de admin sin token de rol	HTTP 403 en 100% de intentos no autorizados	Pasó
Validez del certificado TLS	TLS 1.3, certificado ACM válido, HSTS activo	Pasó
Prueba de replay de QR expirado	HTTP 400 TICKET_EXPIRADO — token invalidado en servidor	Pasó



15. Economía de Ingeniería del Software

15.1. Análisis de Retorno de Inversión (ROI)

Beneficio	Cálculo Base	Valor Anual Estimado (COP)
Reducción de desperdicio alimentario	150 raciones/día × 5 días/sem × 40 sem × \$8.000 c/u	\$ 240.000.000
Ahorro en logística manual de redistribución	2 operarios × 30 min/día × 200 días × salario horario	\$ 12.000.000
Impacto social: acceso a alimentación	150 estudiantes/día beneficiados × valor nutricional estimado	\$ 120.000.000 (equiv. social)
Reducción de quejas y gestión de reclamos	Estimado 60% reducción en tickets de soporte del comedor	\$ 8.000.000
TOTAL BENEFICIOS ANUALES	—	\$ 380.000.000

Concepto	Valor (COP)
Inversión total del proyecto (ver sección 16.2)	\$ 54.863.000
Beneficio bruto año 1	\$ 380.000.000
Beneficio neto año 1 (Beneficio – Inversión)	\$ 325.137.000
ROI año 1 = (Beneficio neto / Inversión) × 100	593%
Período de recuperación (Payback)	< 2 meses

15.2. Presupuesto Base del Proyecto

Rol	Dedicación	Duración	Tarifa/Mes (COP)	Total (COP)
Arquitecto de Software	100%	2 meses	\$ 4.500.000	\$ 9.000.000



Rol	Dedicación	Duración	Tarifa/Mes (COP)	Total (COP)
Desarrollador Backend (Node.js)	100%	4 meses	\$ 3.500.000	\$ 14.000.000
Desarrollador Frontend / Mobile	100%	4 meses	\$ 3.200.000	\$ 12.800.000
Ingeniero DevOps	50%	3 meses	\$ 4.000.000	\$ 6.000.000
QA Engineer	100%	2 meses	\$ 2.800.000	\$ 5.600.000
Diseñador UX/UI	50%	2 meses	\$ 2.500.000	\$ 2.500.000
SUBTOTAL RRHH				\$ 49.900.000

Concepto	Proveedor	Costo Mensual	Meses	Total (COP)
AWS EC2 t3.medium (x2 instancias)	Amazon Web Services	\$ 180.000	4	\$ 720.000
AWS RDS PostgreSQL db.t3.small	Amazon Web Services	\$ 120.000	4	\$ 480.000
AWS ElastiCache Redis cache.t3.micro	Amazon Web Services	\$ 80.000	4	\$ 320.000
AWS ALB + CloudFront	Amazon Web Services	\$ 60.000	4	\$ 240.000
Firebase (FCM plan Blaze)	Google Firebase	\$ 0 (cuota gratuita)	4	\$ 0
SonarQube Community Edition	SonarSource	\$ 0 (open source)	4	\$ 0
GitHub Team Plan	GitHub	\$ 60.000	4	\$ 240.000



Concepto	Proveedor	Costo Mensual	Meses	Total (COP)
Datadog (monitoreo)	Datadog Inc.	\$ 350.000	1	\$ 350.000
SUBTOTAL INFRAESTRUCTURA				\$ 2.350.000

Categoría de Costo	Subtotal (COP)	% del Total
Recursos Humanos	\$ 49.900.000	91.0%
Infraestructura y Licencias	\$ 2.350.000	4.3%
Contingencia (5% del subtotal)	\$ 2.613.000 (aprox.)	4.7%
PRESUPUESTO TOTAL DEL PROYECTO	\$ 54.863.000	100%



16. Bibliografía y Referencias

Brown, S. (2018). *Software architecture for developers: Visualise, document and explore your software architecture*. Leanpub.

Congreso de la República de Colombia. (2012, 17 de octubre). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial N.º 48.587. Imprenta Nacional de Colombia.

Fielding, R., & Reschke, J. (2014). *Hypertext Transfer Protocol (HTTP/1.1): Semantics and content (RFC 7231)*. Internet Engineering Task Force.
<https://tools.ietf.org/html/rfc7231>

Fowler, M. (2012). *Patterns of enterprise application architecture*. Addison-Wesley Professional.

Institute of Electrical and Electronics Engineers. (2008). *IEEE standard for software and system test documentation* (IEEE Std 829-2008).
<https://doi.org/10.1109/IEEESTD.2008.4578383>

Jones, M. B., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT) (RFC 7519)*. Internet Engineering Task Force. <https://tools.ietf.org/html/rfc7519>

Redis Ltd. (2023). *Redis documentation*. <https://redis.io/docs/>

The PostgreSQL Global Development Group. (2023). *PostgreSQL 15 documentation*.
<https://www.postgresql.org/docs/15/>